Fast parameter estimation of generalized extreme value distribution using neural networks

Shu-Ming Chang

March 18 ,2026

Abstract

Motivation

- GEV distribution is widely used for modeling extreme events
- Traditional MLE is computationally expensive

Proposed Approach

- Use neural networks for parameter estimation
- Faster estimation with comparable accuracy to MLE

Abstract

Limitation of Neural Networks

- Neural networks provide point estimates
- No direct measure of uncertainty

Solution: Bootstrap

- Resample data with replacement
- Apply neural network repeatedly
- Obtain distribution of parameter estimates

Introduction

Traditional Estimation Methods

- MLE (Maximum Likelihood Estimation)
  - Accurate but computationally expensive
  - Underestimates tail in small samples
- MOM / PWM / L-moments
  - Faster but less accurate
  - Require subjective weighting

Challenges of GEV Estimation

- Likelihood involves exponential and power functions
- Highly nonlinear structure
- Shape parameter ξ strongly affects tail behavior

Introdution

Why Neural Networks?

- Traditional methods require solving optimization problems
- Neural networks learn direct mapping:

$$X \rightarrow (\mu, \sigma, \xi)$$

- Faster estimation with comparable accuracy

Introdution

Key Idea

- Do not use raw data as input
- Use summary statistics (features) instead

Input Features

- Quantiles
- Quartiles: Q_1, Q_2, Q_3

Introdution

How to do and what is the benifit

- Generate data via simulation
- Train neural network in a supervised manner
- Achieves speedup up to 170×
- Neural network performs direct prediction after training

Uncertainty Estimation

- Neural networks provide point estimates only
- Bootstrap is used to approximate uncertainty
- Fast NN evaluation enables efficient bootstrap sampling

Method : GEV Distribution

Cumulative Distribution Function (CDF)

$$F(x) = \begin{cases} \exp\left(-\left[1+\xi\left(\frac{x-\mu}{\sigma}\right)\right]^{-1/\xi}\right), & \xi \neq 0 \\ \exp\left(-\exp\left(-\frac{x-\mu}{\sigma}\right)\right), & \xi = 0 \end{cases}$$

Support

$$x \in \mathbb{R} : 1+\xi\left(\frac{x-\mu}{\sigma}\right) > 0$$

Distribution Types (by ξ)

- $\xi = 0$: Gumbel (light tail)
- $\xi > 0$: Fréchet (heavy tail)
- $\xi < 0$: Weibull (bounded upper tail)

Method : return level

Definition

$$z_T = F^{-1}(1 - T^{-1})$$

- T: return period
- z_T: level exceeded on average once every T periods

Why Return Level is this form ?

- Because $P(X > z_T) = \frac{1}{T}$ equal to $P(X \leq z_T) = 1 - \frac{1}{T}$ and be reverse it into we interest form.
- use $(1 - T^{-1})$ quantile as the threshold

Why use this form ?

- Because this form contain time ,but if we only suppose the number we don't known the period.

Method:Approximate Sufficient Statistics

Key Idea

- Use quantiles as input features instead of raw data
- Serve as approximate sufficient statistics

Why Quantiles?

- Capture tail behavior of extreme values
- Reduce dimensionality of input data

Approximation

- Not fully sufficient statistics because gev function don't have ss.

Method : Neural Network Training

Training Data

- Simulate data from GEV distribution with known parameters
- Compute quantiles from simulated samples

Learning Process

- Input: quantiles (summary statistics)
- Output: GEV parameters (μ, σ, ξ)
- Neural network learns mapping:

$$T(X) \rightarrow (\mu, \sigma, \xi)$$

Method : GLM vs Neural Networks

Generalized Linear Model

- Linear predictor:

$$\eta = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

- Link function:

$$g(\mu) = \eta$$

Neural Networks

- Input transformed through multiple layers
- Mapping:

$$\text{Input} \rightarrow \text{Hidden Layers} \rightarrow \text{Output}$$

Method : MSE for loss function

Loss Function

$$\text{MSE} = \frac{1}{n_B} \sum_{i=1}^{n_B} \|\theta_i - \hat{\theta}_i\|_2^2$$

Parameter Vector

$$\theta_i = (\mu_i, \sigma_i, \xi_i), \quad \hat{\theta}_i = (\hat{\mu}_i, \hat{\sigma}_i, \hat{\xi}_i)$$

Expanded Form

$$\text{MSE} = \frac{1}{n_B} \sum_{i=1}^{n_B} \left[(\mu_i - \hat{\mu}_i)^2 + (\sigma_i - \hat{\sigma}_i)^2 + (\xi_i - \hat{\xi}_i)^2 \right]$$

Method : Gradient Norm Analysis(I think)

Gradient Contributions

$$G_\mu = \|\nabla_\omega L_\mu\|, \quad G_\sigma = \|\nabla_\omega L_\sigma\|, \quad G_\xi = \|\nabla_\omega L_\xi\|$$

Imbalance Detection

$$G_\xi \ll G_\mu,\ G_\sigma$$

- ξ contributes little to parameter updates
- Training is dominated by μ and σ
- Small gradient $\Rightarrow$ weak learning signal

Method : solve weight of ξ(I think)

Dynamic Weighting Idea

- Adjust weights based on gradient magnitude

$$L = w_\mu L_\mu + w_\sigma L_\sigma + w_\chi L_\chi$$

$$w_\chi \propto \frac{1}{\|\nabla_\omega L_\chi\|}$$

- Balance gradient contributions:

$$\|\nabla_\omega L_\mu\| \approx \|\nabla_\omega L_\sigma\| \approx \|\nabla_\omega L_\chi\|$$

- Automatically prioritizes underlearned parameters

Method : GEV Support Constraint

Support Constraint of GEV

$$1 + \xi \left(\frac{x - \mu}{\sigma} \right) > 0$$

Worst-case Bound

$$\sigma > \max_x \ \xi(\mu - x)$$

- $\xi > 0 \Rightarrow y^* = \min(x)$ and $\xi < 0 \Rightarrow y^* = \max(x)$

$$\Rightarrow \sigma > \xi(\mu - y^*)$$

Reparameterization

$$\sigma = \xi(\mu - y^*) + \exp(\delta)$$

Training : Data Generation

Parameter Sampling

$$\mu \in (1, 50), \quad \sigma \in (0.1, 40), \quad \xi \in (-0.4, 1)$$

Dataset Size

- Training set: 300,000 configurations
- Validation set: 40,000 configurations
- Each configuration generates a GEV sample

Training : Design

Standardization

$$X^* = \frac{X - \text{median}}{\text{IQR}}$$

Sample Size Variation

$$n = 30, 72, 173, 416, 1000$$

Suitable set of percentiles

$$P = \{0.01th, 0.1th, 1th, 10th, 25th, 50th, 75th, 90th, 99th, 99.9th, 99.99th\}$$

Training : Median vs IQR

Standardization Method : IQR vs Standard Deviation

$$X^* = \frac{X - \text{median}}{\text{IQR}} \ , \ X' = \frac{X - \mu}{\sigma}$$

- Extreme values are inherent, not noise
- Mean and standard deviation are sensitive to outliers
- Reflects central tendency without being affected by tails
- Provides stable scaling for neural network training

Method : different layer

TABLE 1 Comparing different architectures trained using various multilayer perceptron (MLP) NNs on the same training set.

No. of hidden layers	Training parameter	Validation loss	Training loss
2-MLP	18,435	21979.52	26,857,216
3-MLP	88,707	4.59	1142.32
5-MLP	614,019	4.58	4.61
6-MLP	1,139,331	4.56	4.54

Note: In this context, "number-MLP" refers to the number of hidden layers in the architecture. After comparing different architectures, the 5-MLP network was chosen for its better performance and efficiency in minimizing the MSE.

Method : structure of NN

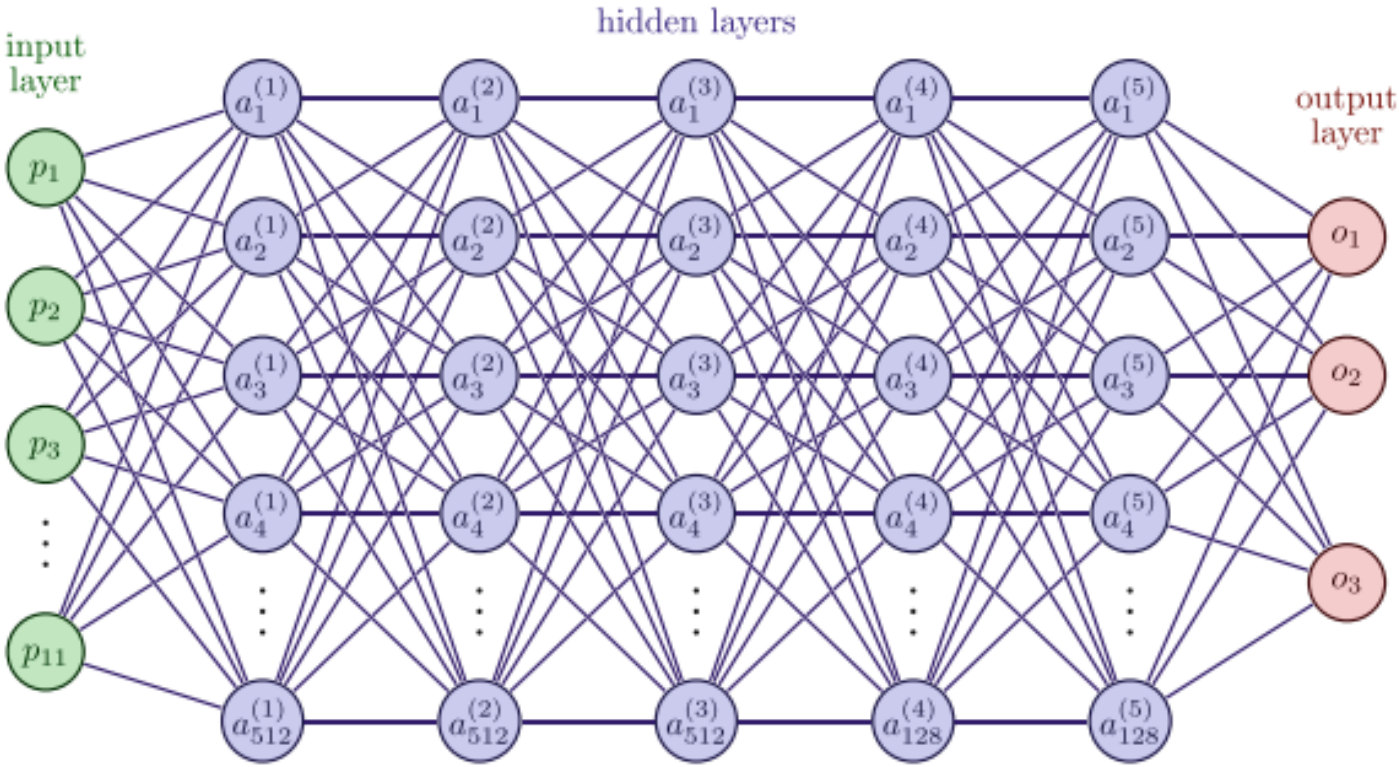


FIGURE 1 The network architecture of $\mathcal{N}$ takes $\mathcal{P}$, denoted as $p_1, p_2, \ldots, p_{11}$, as input. The notation $a_1, \ldots, a_{v_j}$ represents the nodes in the jth hidden layer (h_j), where v_j indicates the number of nodes in the layer, ranging from 1 to 5. The model produces three scalar values as output.

Vanishing Gradient and ReLU Activation

Vanishing Gradient Problem

- During backpropagation, gradients are multiplied across layers
- If derivatives are small (< 1), gradients shrink exponentially

$$\frac{\partial L}{\partial w} = \prod_j \frac{\partial h_j}{\partial h_{j-1}}$$

- Leads to slow or ineffective learning

Why ReLU?

$$\text{ReLU}(x) = \max(0, x)$$

- Derivative:

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

Method : each layer parameter and activation

TABLE 2 Overview of hidden layers and output layer in our FFNN model.

Layer type	Output shape	Activation	Parameters
Dense	512	ReLU	6144
Dense	512	ReLU	262,656
Dense	512	ReLU	262,656
Dense	128	ReLU	65,664
Dense	128	ReLU	16,512
Dense	3	Linear	387

Note: The input layer has shape 11; 11 quantile values as inputs, and returning the estimated GEV parameters as output with shape 3.

Method : reverse Standardization

Standardization

$$z = \frac{y - \tilde{y}}{\mathrm{IQR}}$$

Effect on Parameters

$$\mu^* = \frac{\mu - \tilde{y}}{\mathrm{IQR}}, \quad \sigma^* = \frac{\sigma}{\mathrm{IQR}}, \quad \xi^* = \xi$$

Motivation

- Removes scale and unit dependence
- Improves training stability
- Robust to extreme values

Inverse Transformation

$$\mu = \mu^* \cdot \mathrm{IQR} + \tilde{y}, \quad \sigma = \sigma^* \cdot \mathrm{IQR}$$

Method : RMSE of σ

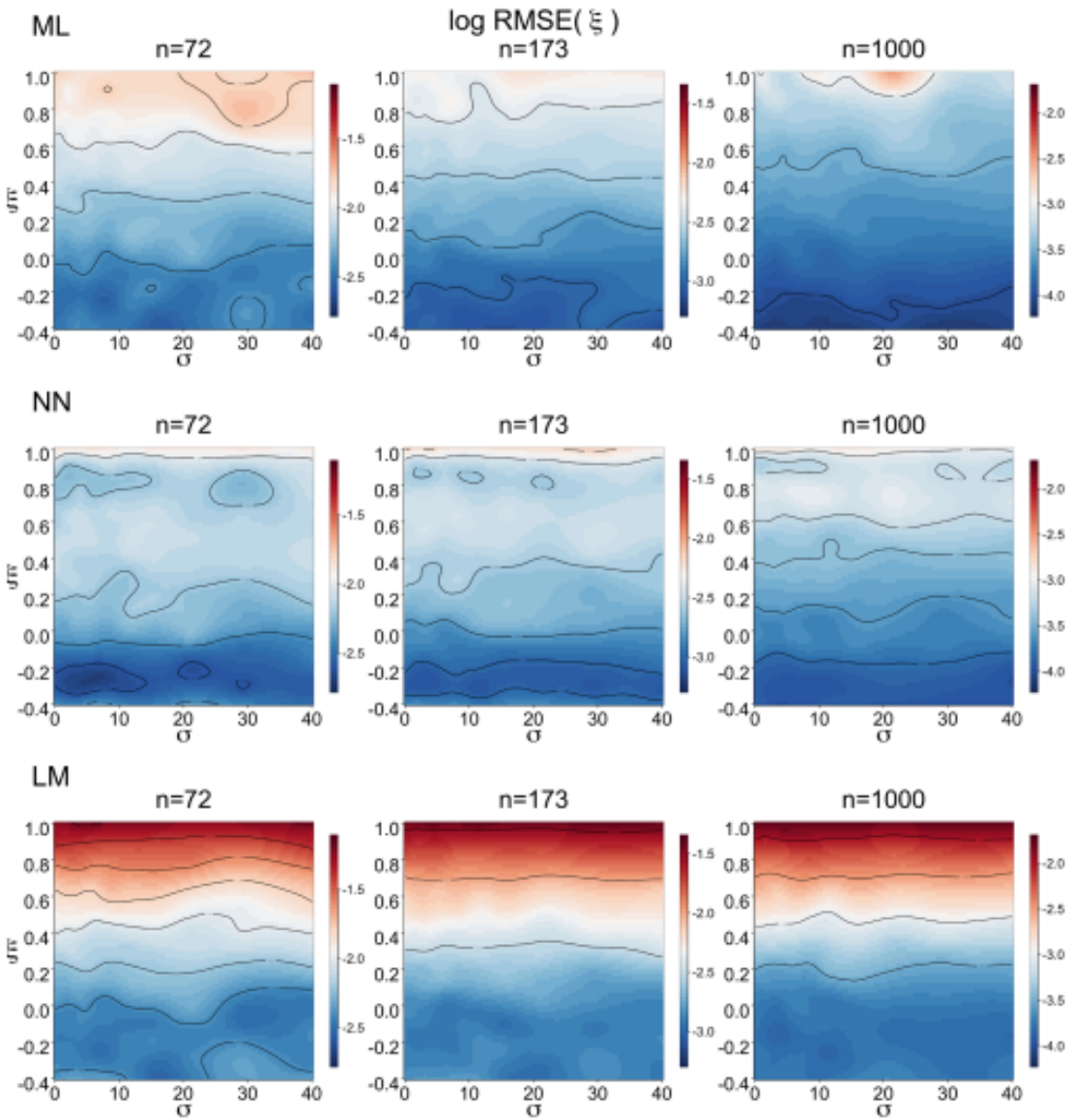


FIGURE 2 This figure illustrates the behavior of the RMSE(ξ) as the sample size varies among 72, 173, 1000 based on our simulation study for the ML, the NN (neural estimator), and the LM approach. The image plot showcases the logarithmic RMSE values concerning the true (σ, ξ), with σ on the x-axis and ξ on the y-axis, displaying the outcomes of each approach in separate rows.

Method : RMSE of ξ

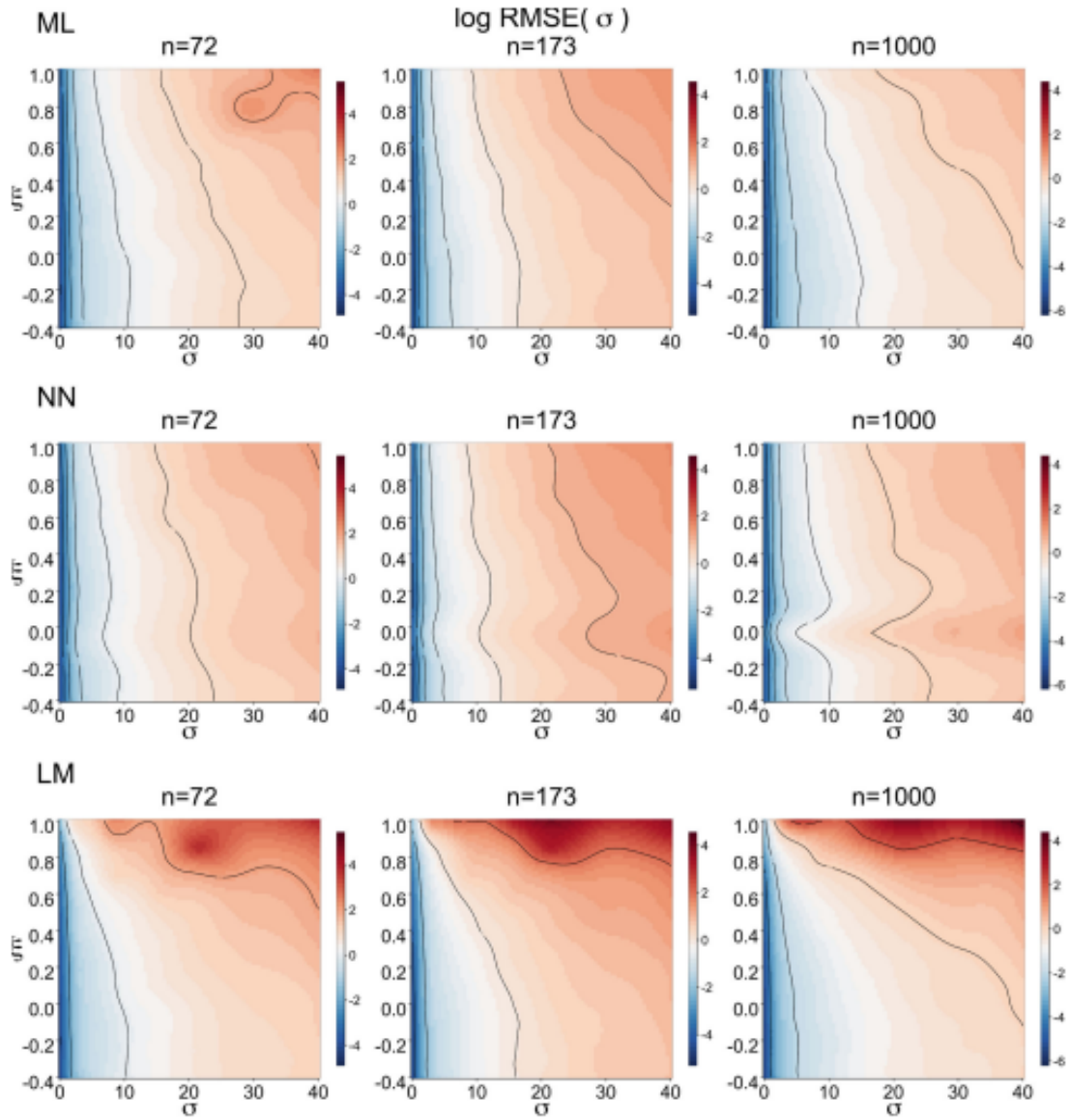


FIGURE 3 This figure illustrates the behavior of the RMSE(σ) as the sample size varies among 72, 173, 1000 based on our simulation study for the ML, the NN (neural estimator), and the LM approach. The image plot showcases the logarithmic RMSE values concerning the true (σ, ξ), with σ on the x-axis and ξ on the y-axis, displaying the outcomes of each approach in separate rows.

Method : Bootstrap for CI

Bootstrap Procedure

- Step 1: Take $B = 1000$ resample from data

$$X = \{x_1, \ldots, x_n\} \rightarrow X_1^*, \ldots, X_B^*$$

- Step 2: Estimate parameters using NN

$$(\hat{\theta}_1, \ldots, \hat{\theta}_B)$$

- Step 3: Construct confidence interval

$$CI = [2.5\%, 97.5\%]$$

Method : Evaluation of Bootstrap CI

Likelihood-based CI

- Derived from Hessian matrix of MLE
- Uses asymptotic normal approximation

Experimental Design

- Parameter range: $-0.4 < \xi \leq 0.8$
- 25 parameter configurations
- Sample size: $n = 416$
- 100 repetitions per configuration

Evaluation Metrics

- CI width (precision)
- Coverage probability (accuracy)

Method : bootstrap with NN of CI

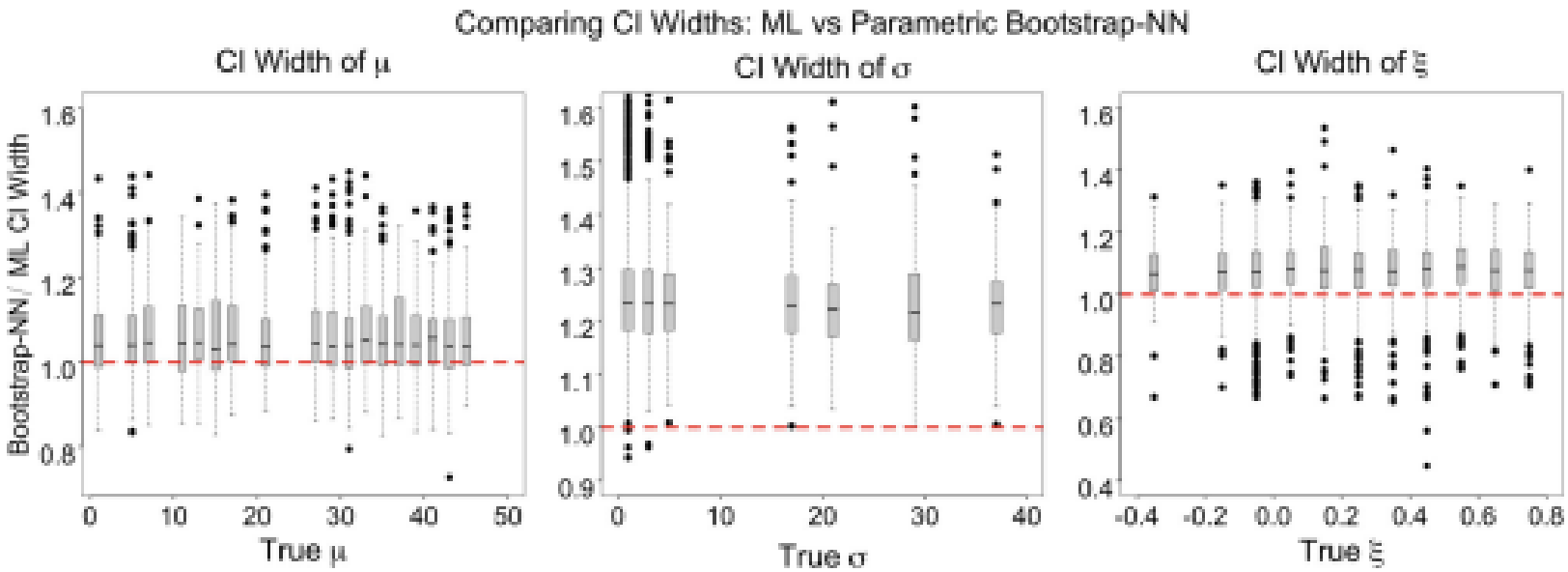


FIGURE 4 The figure compares the CI widths obtained from the Bootstrap-NN and ML approaches for GEV parameters across the true parameter values.

Method : RMSE of ξ

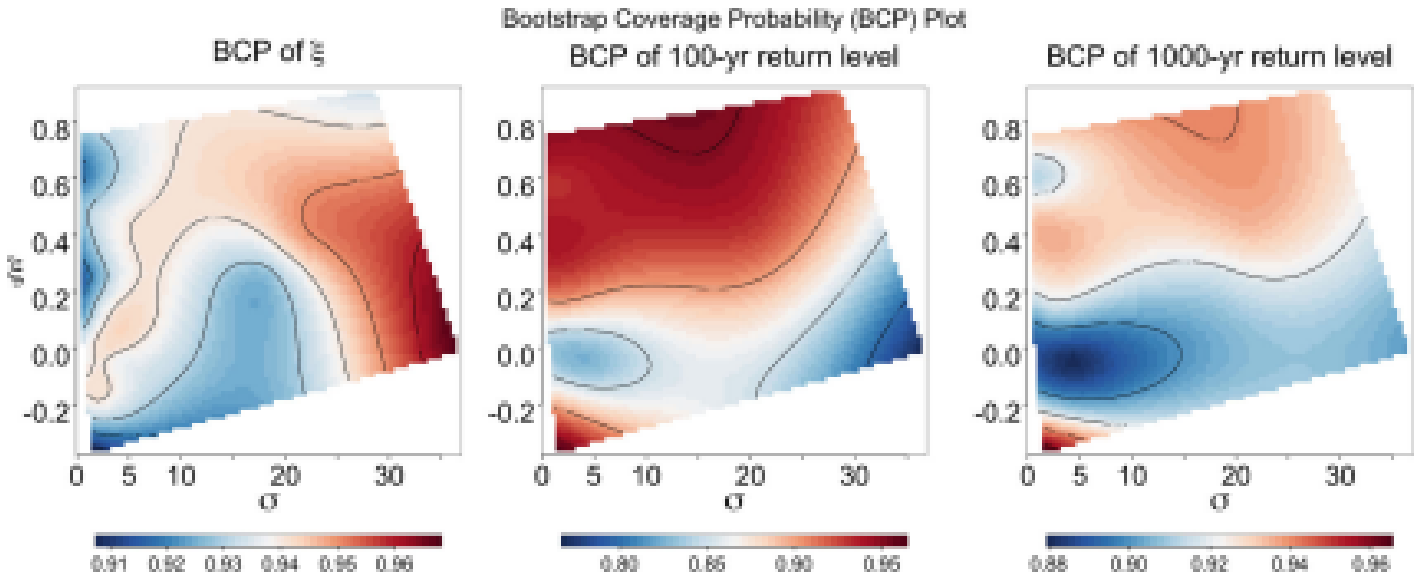


FIGURE 5 Surface plot depicting the bootstrap coverage probability for ξ, the 100-year return level, and the 1000-year return level, plotted against the true σ and ξ parameters.

Compare time of NN and ML

Neural Network

- Training: ≈ 0.21 hours (one-time cost)
- Inference: fast forward pass (matrix operations)

MLE / L-moment

- Requires optimization for each dataset
- Computationally expensive

Results

- Tested on 10,000 parameter configurations
- Speedup $> 170\times$ compared to ML

Need to do or fix

- reproduce the method
- use Check that the method betweeen 15-16 can enhance the old method or not.

Case Study : backgroung/data

Main Idea

- Use trained Neural Network to estimate GEV parameters (μ, σ, ξ)
- Analyze how increasing CO_2 affects extreme temperature distributions

Data Source

- CCSM3 climate model simulation
- 1000-year annual maximum temperature (block maxima)
- 133 spatial locations across North America

Statistical Framework

- Use EVT transform to GEV distribution and modeled as

$$Y_{\max} \sim \text{GEV}(\mu, \sigma, \xi)$$

Case Study :estimate time

TABLE 3 Comparing the evaluation time across different sample sizes.

Estimation approach	72	173	1000
ML	2.92 min	5.81 min	11.66 min
L-moments	22.32 s	27.64 s	1.79 min
Neural estimator	2 s	3 s	4 s

Case Study : NN estimate

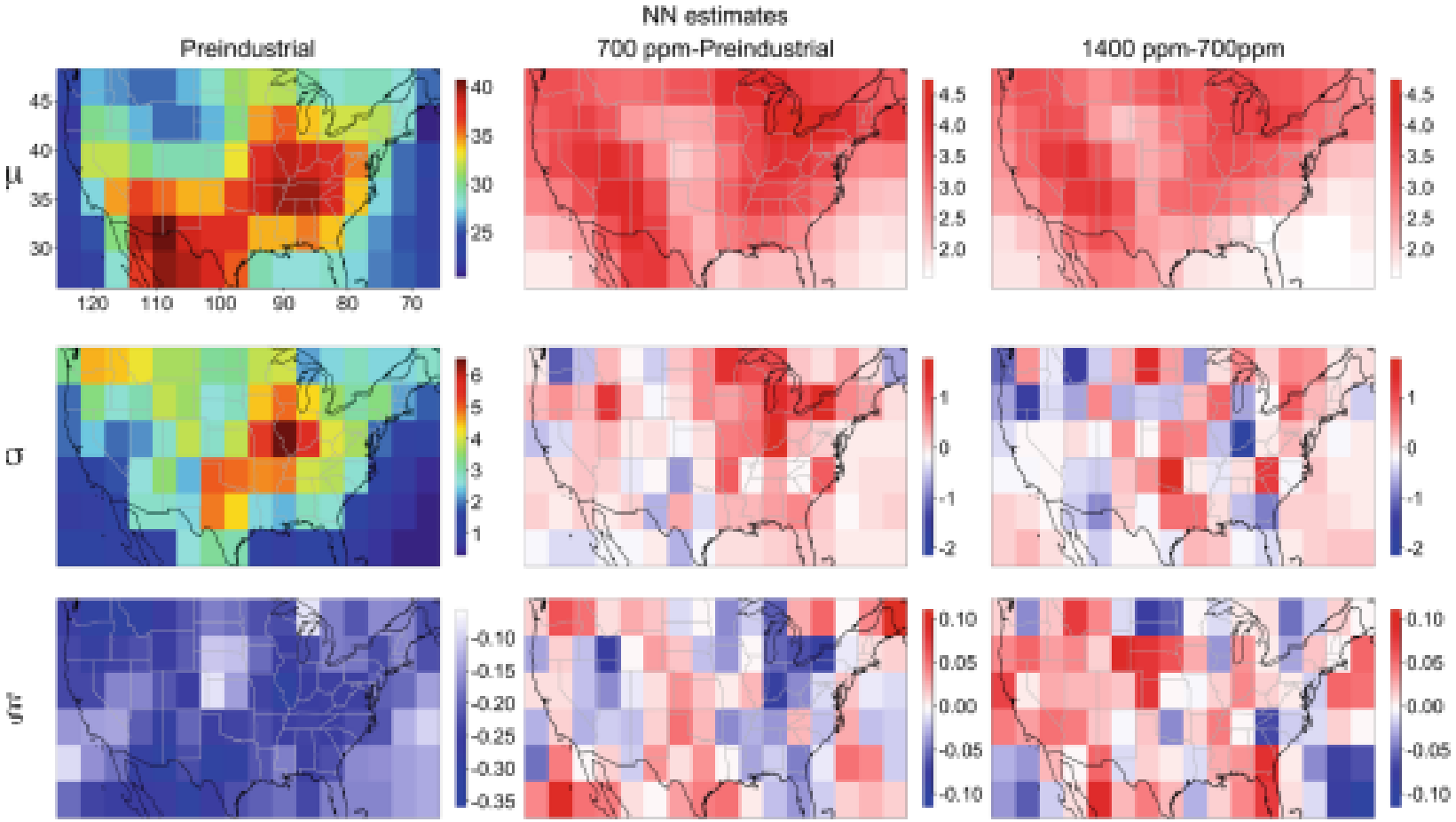


FIGURE 6 NN estimates of CCSM3 GEV parameter for the pre-industrial period and possible changes for future cases. On the left: are estimates for the pre-industrial, center: the expected change in parameter estimates moving from 289 ppm CO_2 to 700 ppm CO_2 concentration, and right: is the change in estimates for 700 ppm CO_2 to 1400 ppm.

Method : RMSE of ξ

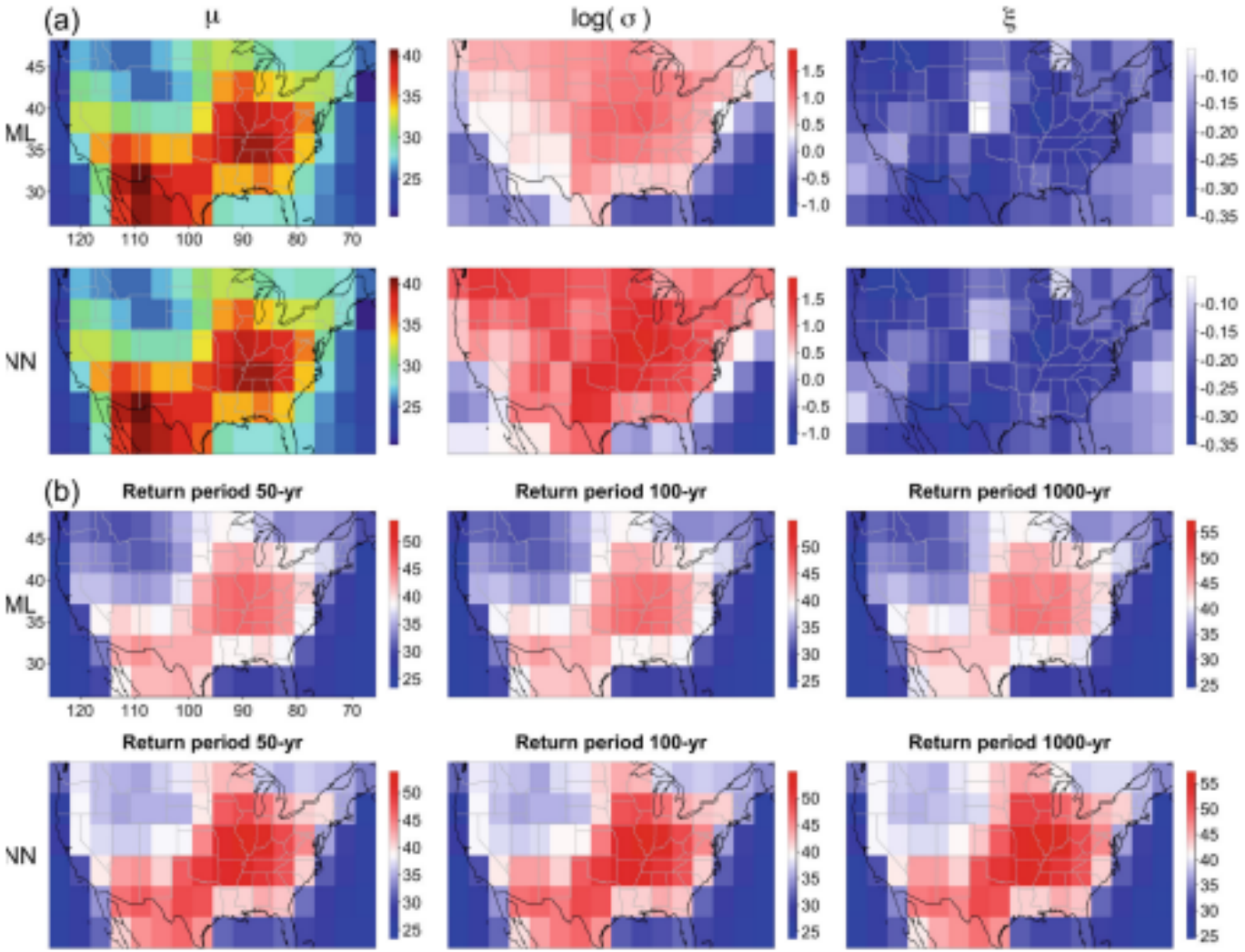


FIGURE 7 (a) Show the CCSM3 GEV parameter estimates from the ML and the neural estimator $\mathcal{N}$ across the 133 spatial locations for pre-industrial concentration scenarios. (b) Comparison plots of return levels for 50-, 100-, and 1000-year periods between the ML approach and $\mathcal{N}$, focusing on pre-industrial concentration scenario.

Conclusion

Main Contributions

- Neural Networks (NN) can estimate GEV parameters accurately
- Performance comparable to Maximum Likelihood (ML)
- Significant computational speed-up (up to 170x faster)
- Flexible training across wide parameter ranges

Limitations

- Hyperparameter tuning is complex
- Large model size and training cost
- Requires careful selection of input statistics (quantiles)
- Outputs may violate constraints without reparameterization

Future Work

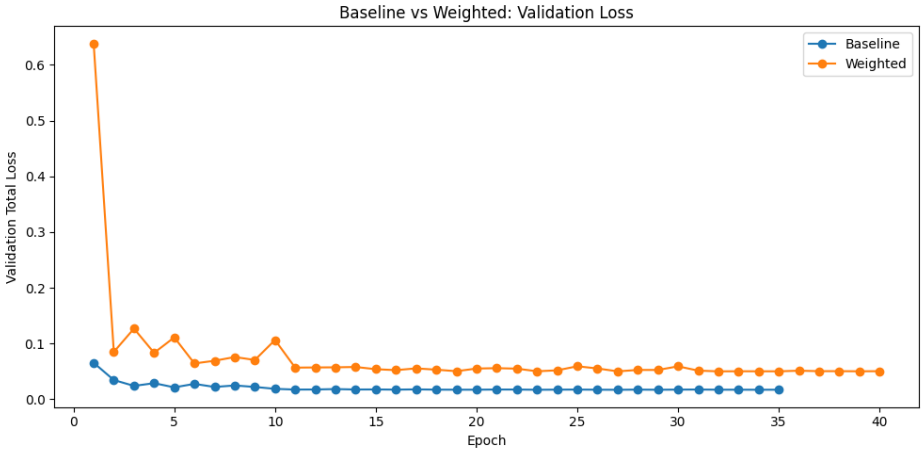
- Time-dependent GEV modeling
- GEV regression and GPD extension
- Direct estimation of return levels
- Spatial extreme value modeling

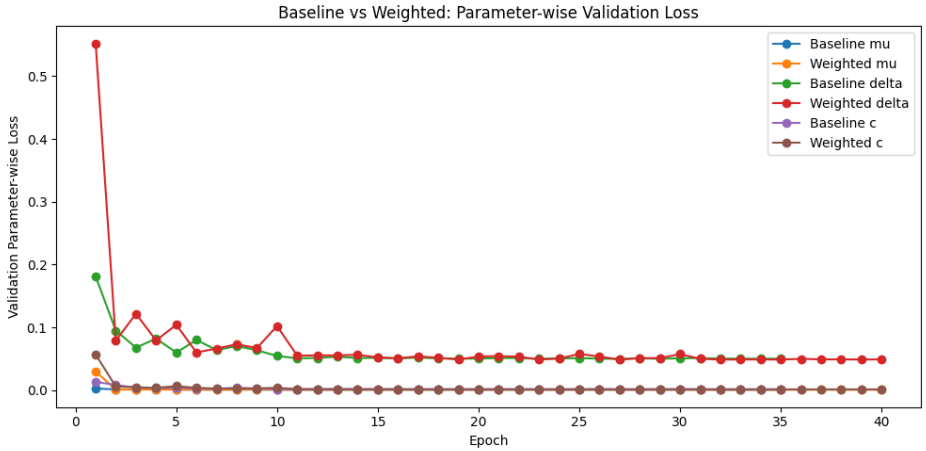
Reproduce: Baseline vs. Weighted

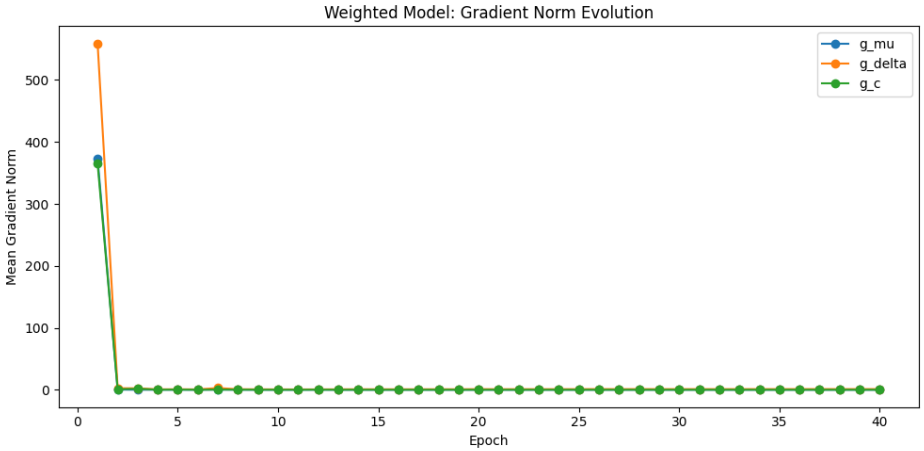
Metric	Baseline	Weighted	Interpretation
val_total	**0.017266**	0.050503	overall loss increases
val_mu	0.000531	**0.000463**	improved
val_delta	0.050063	**0.048904**	slightly improved
val_xi	0.001205	**0.001136**	improved

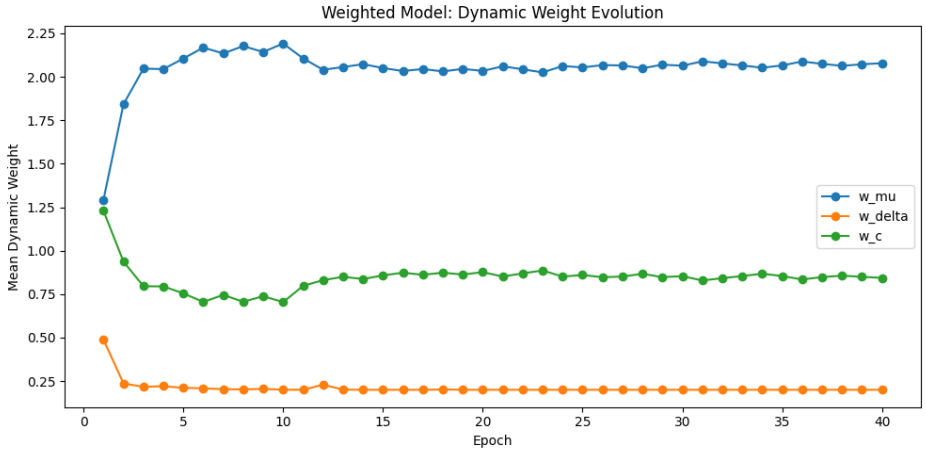
- The validation loss is dominated by the δ component.
- But the ξ is more important for the result.

Reproduce: Baseline vs. Weighted







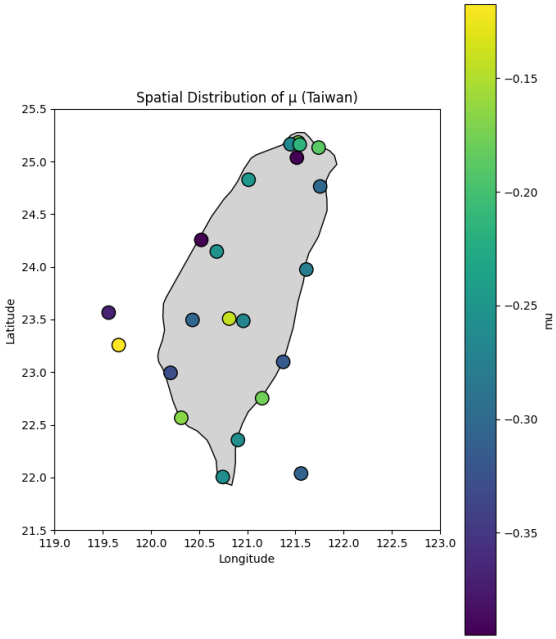


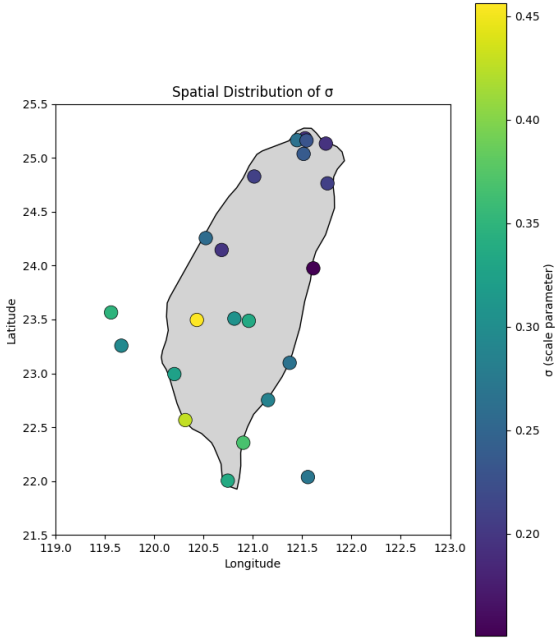
Reproduce: real data

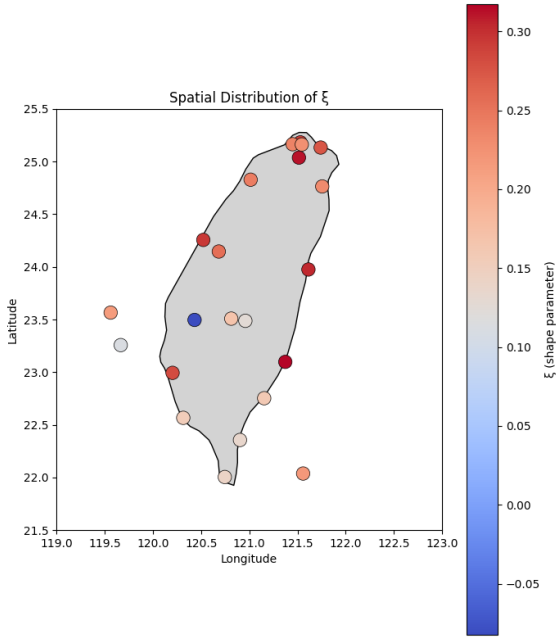
Date	ALISHAN	ANBU	CHENGGONG	CHIAYI	DAWU	DONGJIDAO	HENGCHUN	\
1980-12-31	14.8	24.0	28.3	29.0	29.2	28.5	29.4	
1981-12-31	14.6	23.4	28.7	28.5	29.2	28.1	28.8	
1982-12-31	14.7	22.8	28.2	28.3	28.5	27.6	27.9	
1983-12-31	15.0	23.9	29.0	29.6	29.7	29.4	29.2	
1984-12-31	14.4	23.8	28.0	28.4	28.4	28.0	28.1	
1985-12-31	14.1	22.4	27.7	28.2	28.0	27.8	27.7	
1986-12-31	14.2	23.0	28.1	28.7	28.5	28.1	27.7	
1987-12-31	14.5	23.4	28.3	28.8	28.8	28.7	28.8	
1988-12-31	14.6	23.8	28.6	28.7	28.6	28.4	28.5	
1989-12-31	14.3	23.1	28.1	28.4	28.4	28.6	28.8	
1990-12-31	14.6	23.1	28.1	28.5	29.0	28.2	28.3	
1991-12-31	14.7	23.5	28.8	28.5	29.4	28.4	28.5	
1992-12-31	14.3	23.3	28.1	28.2	29.1	28.1	28.6	
1993-12-31	14.6	23.5	28.4	29.0	29.4	28.6	28.8	
1994-12-31	14.8	23.2	28.5	28.4	28.2	28.3	28.3	
1995-12-31	14.4	22.9	27.0	28.3	28.6	27.6	28.3	
1996-12-31	14.7	23.2	28.1	28.9	28.6	28.6	28.9	
1997-12-31	14.5	22.6	27.3	28.2	27.7	28.1	28.0	

- the shape of real data is (45*25)

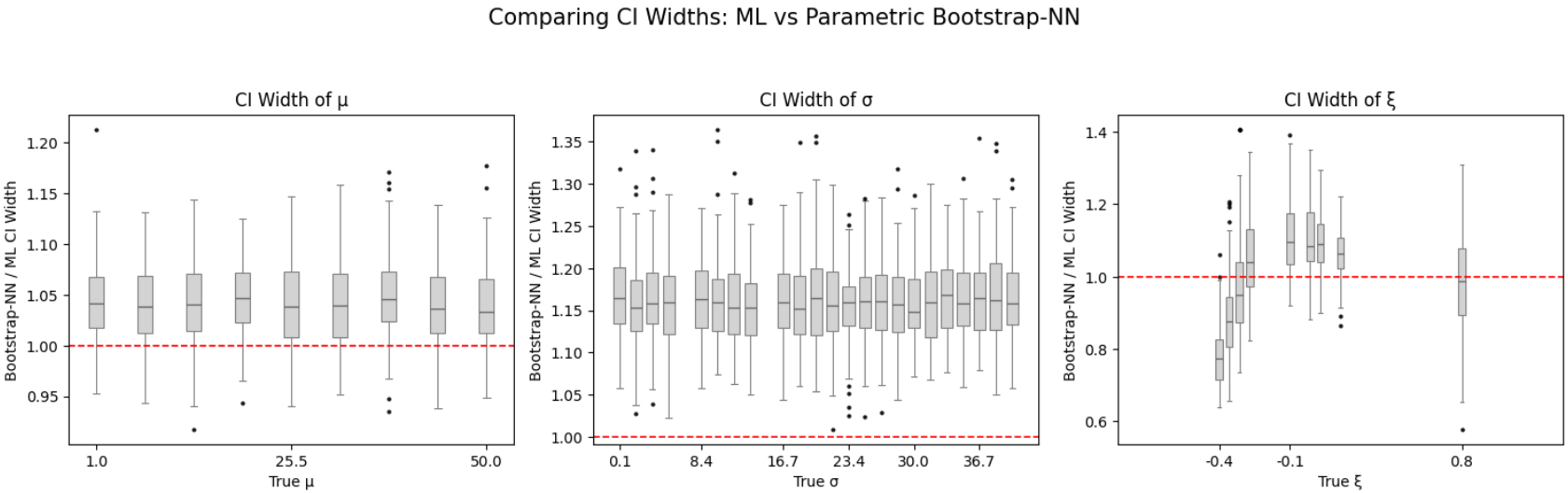
Reproduce: preediction value use 25 stations







Reproduce: ci interval



Need to do or fix

- Think if the parameter is random process, What is the way to estimate it and how to put into ML and NN

Original Paper: What Does the NN Estimate?

$$\text{Sample quantiles} \quad \longrightarrow \quad (\hat{\mu}, \hat{\sigma}, \hat{\xi})$$

- Input: selected sample quantiles from block maxima(11 quantile).
- Output: point estimates of GEV parameters.
- The model is applied independently at each location.
- It does not explicitly model spatial dependence among locations.

Why Does the Case Study Look Spatial?

For each location s_i,

$$Y_t(s_i) \sim GEV(\mu(s_i), \sigma(s_i), \xi(s_i))$$

and the NN estimates

$$\hat{\theta}(s_i) = (\hat{\mu}(s_i), \hat{\sigma}(s_i), \hat{\xi}(s_i)).$$

- Each location has its own annual maximum time series.
- The NN estimates GEV parameters separately for each location.
- The estimated parameters are then plotted on a map.

If Parameters Are Random Processes

Original Assumption

The original model treats the GEV parameters at each site as fixed unknown quantities.

$$Y_t(s_i) \sim GEV(\mu_i, \sigma_i, \xi_i)$$

Modified Assumption

Assume the three GEV parameters vary over space as random processes.

$$Y_t(s) \mid \mu(s), \sigma(s), \xi(s) \sim GEV(\mu(s), \sigma(s), \xi(s))$$

$$\mu(s), \quad \log \sigma(s), \quad \xi(s)$$

are spatial random processes.

Proposed Modification: Spatial Parameter Process Model

$$\mu(s) = x(s)^\top \beta_\mu + W_\mu(s)$$

$$\log \sigma(s) = x(s)^\top \beta_\sigma + W_\sigma(s)$$

$$\xi(s) = x(s)^\top \beta_\xi + W_\xi(s)$$

where

$$W_\mu(s), \quad W_\sigma(s), \quad W_\xi(s)$$

are spatial Gaussian processes.

- $\mu(s)$: spatial process for location parameter.
- $\log \sigma(s)$: ensures the scale parameter is positive.
- $\xi(s)$: spatial process for tail behavior.

What Does $\mu(s) = x(s)^\top \beta_\mu + W_\mu(s)$ Mean?

$$\mu(s) = x(s)^\top \beta_\mu + W_\mu(s)$$

- $\mu(s)$: the true but unknown GEV location parameter at location s
- $x(s)^\top \beta_\mu$: large-scale spatial trend explained by covariates
- $W_\mu(s)$: spatial random effect for local dependence

Interpretation

The parameter at location s is decomposed into:

$$\text{parameter} = \text{explained trend} + \text{spatially correlated residual part}$$

What Does $\mu(s) = x(s)^\top \beta_\mu + W_\mu(s)$ Mean?

For example, if

$$x(s) = \begin{pmatrix} 1 \\ \text{longitude}(s) \\ \text{latitude}(s) \\ \text{elevation}(s) \end{pmatrix},$$

then

$$x(s)^\top \beta_\mu = \beta_0 + \beta_1 \text{longitude}(s) + \beta_2 \text{latitude}(s) + \beta_3 \text{elevation}(s).$$

And

$$W_\mu(s) \sim GP(0, C_\mu(h))$$

Two-Stage Framework: NN + Kriging

- Use the trained NN to estimate GEV parameters at observed stations.

$$\hat{\theta}(s_i) = (\hat{\mu}(s_i), \widehat{\log \sigma}(s_i), \hat{\xi}(s_i))$$

- Treat NN estimates as noisy observations of latent spatial processes.

$$\hat{\mu}(s_i) = \mu(s_i) + \epsilon_{\mu,i}$$

$$\widehat{\log \sigma}(s_i) = \eta(s_i) + \epsilon_{\sigma,i}$$

$$\hat{\xi}(s_i) = \xi(s_i) + \epsilon_{\xi,i}$$

$$\eta(s) = \log \sigma(s)$$

Final Output: Spatial Extreme-Value Distribution

Prediction at an Unobserved Location

After kriging, we can estimate the GEV parameters at any location s_0.

$$\hat{\mu}(s_0), \quad \hat{\sigma}(s_0), \quad \hat{\xi}(s_0)$$

Then the local extreme-value distribution is

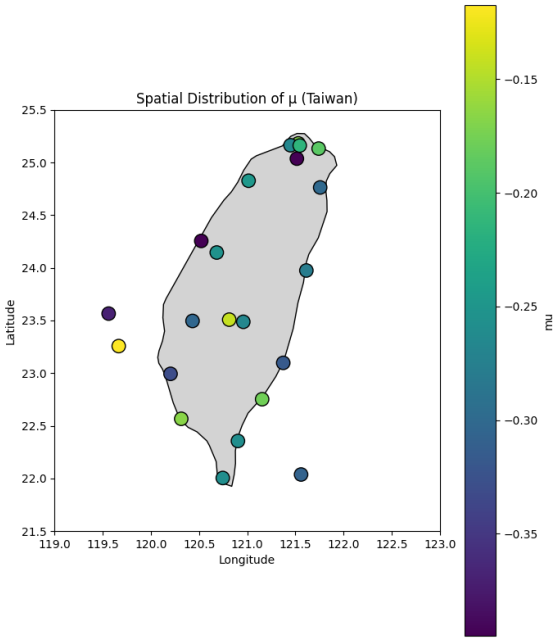
$$Y_t(s_0) \sim GEV(\hat{\mu}(s_0), \hat{\sigma}(s_0), \hat{\xi}(s_0)).$$

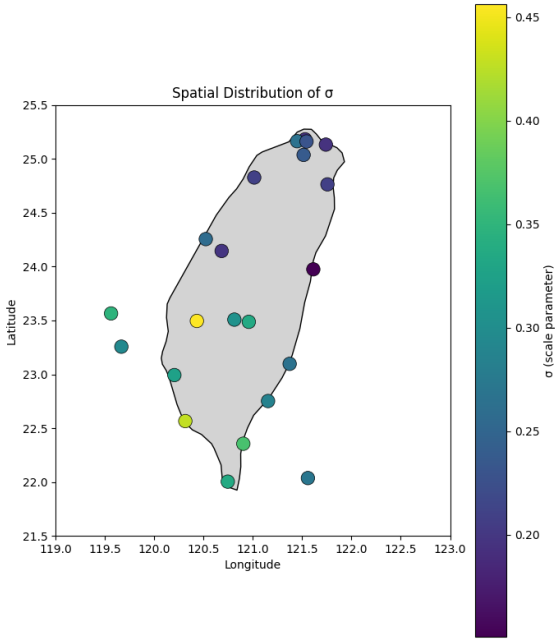
Return Level Map

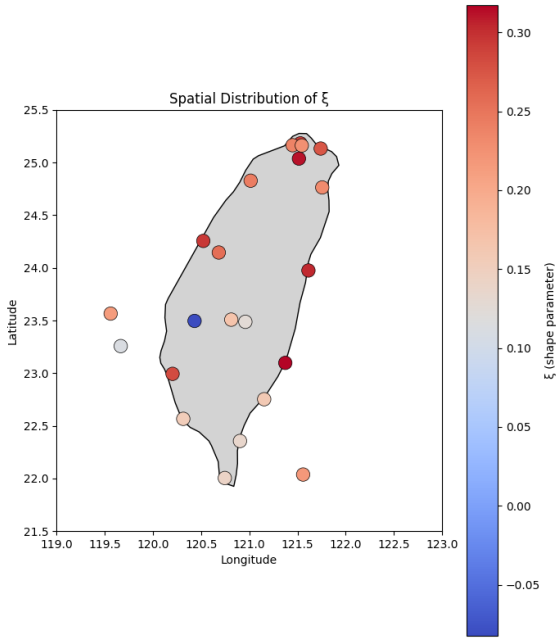
For return period T, the return level surface is

$$z_T(s) = \mu(s) + \frac{\sigma(s)}{\xi(s)} \left[\{-\log(1 - 1/T)\}^{-\xi(s)} - 1 \right].$$

Reproduce: preediction value use 25 stations







real data use kriging